

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. Vector fields are used in various scientific and engineering disciplines to represent quantities that have both magnitude and direction at every point in space. In the context of computer graphics and particle systems, vector fields can influence the motion of particles to simulate natural phenomena like wind or water currents. Which of the following best describes a vector field?

1 / 1 point

- ☐ A Collection of Discrete Points
- ☐ A Grid Assigning a Single Value to Each Point
- ☒ A Spatial Representation Where Each Point Has a Vector.
- ☐ An Array of Vectors With Fixed Orientation
- ☐ A Mathematical Function That Outputs the Same Vector Everywhere

✔ Correct
Correct. This description accurately captures the essence of a vector field. A vector field is indeed a spatial representation where each point in the space is associated with a vector, which specifies a direction and a magnitude. This concept is crucial in simulating forces that act on particles in computer graphics and particle systems.

2. In a Processing sketch using Python mode, you want to create a vector field where each vector points in a random direction. This vector field will influence the movement of particles, simulating an environment with varying forces. How would you initialize a two-dimensional array (grid) to represent this vector field, ensuring each vector within it has a random direction?

1 / 1 point

✔ ☒

```
1 import random
2
3 fieldSize = 10
4 vectorField = []
5 for i in range(fieldSize):
6     row = []
7     for j in range(fieldSize):
8         x = random.uniform(-1.0, 1.0)
9         y = random.uniform(-1.0, 1.0)
10        direction = PVector(x, y)
11        row.append(direction)
12    vectorField.append(row)
13
```

☐

```
1 fieldSize = 10
2 vectorField = []
3 for i in range(fieldSize):
4     row = []
5     for j in range(fieldSize):
6         row.append(PVector(1, 0))
7     vectorField.append(row)
8
```

☐

```
1 fieldSize = 10
2 vectorField = []
3 for i in range(fieldSize):
4     row = []
5     for j in range(fieldSize):
6         row.append(random.uniform(-1, 1))
7     vectorField.append(row)
8
```

☐

```
1 fieldSize = 10
2 vectorField = []
3 for i in range(fieldSize * fieldSize):
4     vectorField.append(PVector.random2D())
5
```

☐

```
1 fieldSize = 10
2 vectorField = []
3 for i in range(fieldSize)
4     row = []
5     for j in range(fieldSize):
6         row.append(PVector.random2D())
7     vectorField.append(row)
8
```

✔ Correct
Correct. This code uses random.uniform(-1.0, 1.0) to generate random floating-point numbers between -1.0 and 1.0 for both the X and Y components of each vector. This range ensures that the vectors can point in any direction in the 2D plane.

3. In a Processing sketch using Python mode, you've created a vector field represented as a two-dimensional grid (array) of vectors. The grid spans the entire canvas, which is divided into a fixed number of columns and rows, creating cells that each contain a vector. You want to use the mouse position to interact with this grid, specifically to identify which cell (and thus, which vector) the mouse is pointing to at any given moment. Given that the canvas size is width by height pixels and the grid size is gridWidth by gridHeight cells, how can you convert the mouse's continuous screen coordinates (mouseX, mouseY) into discrete grid indices to access the corresponding vector in the vector field?

1 / 1 point

☐

```
1 columnIndex = mouseX
2 rowIndex = mouseY
```

✔ ☒

```
1 columnIndex = int(mouseX / (width / gridWidth))
2 rowIndex = int(mouseY / (height / gridHeight))
```

☐

```
1 columnIndex = mouseX / width * 100
2 rowIndex = mouseY / height * 100
```

☐

```
1 columnIndex = mouseY / (height / gridWidth)
2 rowIndex = mouseX / (width / gridHeight)
```

☐

```
1 columnIndex = mouseX * gridWidth
2 rowIndex = mouseY * gridHeight
```

✔ Correct
Correct. This solution correctly maps the mouse's screen coordinates to the vector field's grid indices by dividing the canvas into segments based on the number of grid cells and using integer division to ensure the indices are discrete values. This method effectively translates continuous mouse positions into specific cells within the grid.